

TaskGuard

Implementação de um Sistema Web Seguro de Gerenciamento de Tarefas com Pipeline DevSecOps

Relatório Acadêmico • Engenharia DevSecOps • Python • Flask • Docker • PostgreSQL • OWASP

1. Introdução

A crescente frequência de ataques a aplicações web tornou insustentável tratar a segurança como uma etapa posterior ao desenvolvimento. A cultura *DevSecOps* responde a esse cenário integrando segurança de forma contínua e automatizada ao ciclo de vida do software (*shift-left security*). Este relatório descreve o **TaskGuard**, um sistema web de gerenciamento de tarefas pessoais desenvolvido em Python/Flask cujo objetivo principal é servir de estudo de caso prático e completo de DevSecOps, contemplando autenticação robusta, defesas contra o OWASP Top 10, persistência em PostgreSQL, containerização, observabilidade e uma *pipeline* de integração contínua que reprova automaticamente código e dependências inseguros.

2. Desenvolvimento

A aplicação foi construída sobre o microframework **Flask 3**, adotando o padrão *application factory* e *blueprints* para modularizar os domínios de autenticação e de tarefas. A persistência utiliza o ORM **SQLAlchemy** sobre um banco **PostgreSQL 16**, eliminando a construção manual de SQL. As funcionalidades entregues incluem cadastro e *login* obrigatórios, CRUD completo de tarefas com busca e filtros, marcação de conclusão e isolamento total entre usuários. A camada de apresentação emprega *templates* Jinja2 com *auto-escape* e uma interface própria de tema escuro inspirada em consoles de operações de segurança.

O ecossistema de ferramentas combina **Docker** (imagem *multi-stage* e execução sem privilégios), **Docker Compose** (banco PostgreSQL, aplicação, coletor de logs e ZAP), **GitLab CI/CD** (orquestração da *pipeline*) e a tríade de verificação de segurança **Bandit** (SAST), **OWASP Dependency-Check** (SCA) e **OWASP ZAP** (DAST), além de `pytest` e `flake8` para qualidade.

3. Arquitetura

A arquitetura segue o princípio de *defesa em profundidade*: o tráfego atravessa camadas independentes de proteção antes de alcançar a lógica de negócio. O *middleware* ProxyFix recupera o IP real do cliente; o Flask-Talisman aplica CSP, HSTS e demais cabeçalhos; o Flask-Limiter impõe limites de requisição; e o Flask-WTF valida *tokens* CSRF. Apenas então as rotas dos *blueprints* executam, sempre mediadas pelo ORM. Os principais componentes estão resumidos a seguir.

Camada	Componentes	Responsabilidade
Borda	Proxy reverso, ProxyFix	Terminação TLS e identificação do cliente
Segurança	Talisman, Limiter, Flask-WTF	Cabeçalhos, <i>rate limiting</i> e CSRF
Aplicação	<i>Blueprints</i> auth e tasks	Autenticação, CRUD e autorização
Dados	SQLAlchemy ORM + PostgreSQL 16	Acesso parametrizado (anti-SQLi)
Observabilidade	syslog-ng, Fail2Ban, monitor	Logs centrais e detecção de <i>brute force</i>

4. Pipeline DevSecOps

A *pipeline*, definida em **GitLab CI/CD** (arquivo `.gitlab-ci.yml`), encadeia sete *jobs* distribuídos em seis *stages* e interrompe a entrega ao primeiro indício de risco. Os *jobs* de análise de segurança (`sast` e `dependency-check`) executam em paralelo no *stage security*, reduzindo o tempo de *feedback*.

Estágio	Ferramenta	Critério de falha
lint	flake8	Violação de estilo ou complexidade
test	pytest + coverage	Teste quebrado ou cobertura < 80%
sast	Bandit	Vulnerabilidade de severidade ALTA
dependency-check	OWASP Dependency-Check	Dependência com CVSS ≥ 7
docker-build	Docker-in-Docker	Falha de <i>build</i> da imagem
dast	OWASP ZAP	Alerta classificado como FAIL
deploy-stage	—	Promoção a <i>staging</i> (<i>branch main</i>)

5. Segurança

A modelagem de ameaças baseou-se em **STRIDE** e no **OWASP Top 10**. Para cada ameaça relevante implementou-se uma mitigação concreta e verificável, conforme a tabela abaixo (amostra dos principais controles).

Ameaça	Mitigação implementada
SQL Injection	Acesso exclusivo via ORM com consultas parametrizadas
XSS	Auto-escape Jinja2 + sanitização + CSP estrita com <i>nonce</i>
CSRF	Token CSRF (Flask-WTF); ações destrutivas apenas via POST
Brute force	Rate limiting + bloqueio temporário de conta + senha forte
Broken Access Control (IDOR)	Validação de proprietário por tarefa; acesso indevido gera 404
Sequestro de sessão	Cookies HttpOnly/Secure/SameSite e <i>session protection strong</i>
Exposição de credenciais	Hash PBKDF2-SHA256; segredos em variáveis de ambiente
Dependências vulneráveis	OWASP Dependency-Check na <i>pipeline</i> (falha em CVSS ≥ 7)

6. Resultados

A aplicação foi executada e validada com sucesso, inclusive contra um banco **PostgreSQL** real (criação de *schema*, persistência via ORM, *hash* de senha e bloqueio por *brute force* confirmados). A suíte automatizada passou integralmente e as defesas de segurança foram verificadas em resposta HTTP real (cabeçalhos presentes, CSRF ativo, XSS neutralizado). O quadro a seguir resume os resultados obtidos.

Verificação	Resultado
Testes automatizados (pytest)	29 testes — 100% aprovados
Cobertura de código	86,65% (mínimo exigido: $\geq 80\%$)
Análise estática (Bandit)	Nenhuma <i>issue</i> de severidade ALTA
Cabeçalhos de segurança	CSP, HSTS, X-Frame-Options, <i>nosniff</i> presentes
Proteção CSRF	POST sem <i>token</i> rejeitado (HTTP 400)
Proteção contra <i>brute force</i>	Conta bloqueada após N tentativas
Health check	HTTP 200 com verificação do banco

7. Evidências (prints)

As capturas de tela do sistema em execução e dos relatórios das ferramentas de segurança devem ser inseridas nas molduras abaixo. Para cada uma, substitua o conteúdo por um comando `includegraphics` apontando

para o arquivo do print (sugestão: mantenha as imagens em docs/screenshots/).

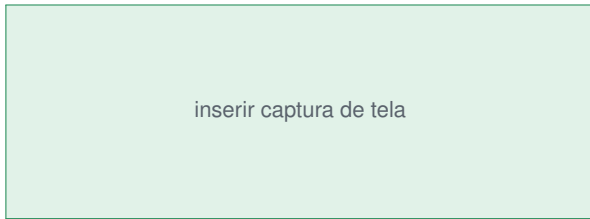


Figura 1. Tela de login

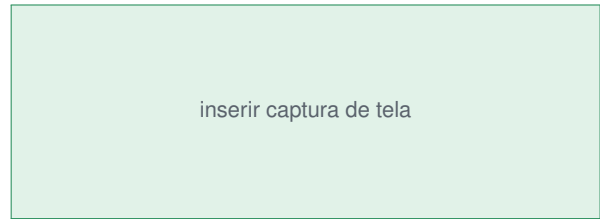


Figura 2. Painel de tarefas

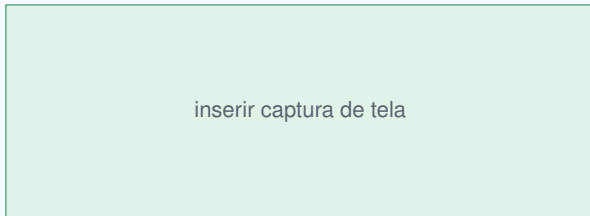


Figura 3. Relatório do OWASP ZAP (DAST)

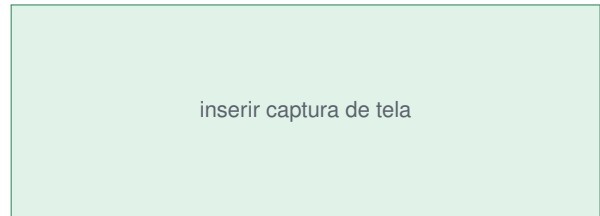


Figura 4. Pipeline no GitLab CI/CD

8. Conclusão

O TaskGuard demonstra, na prática, que a segurança pode permear todo o ciclo de desenvolvimento sem comprometer a entrega. A combinação de um *backend* bem estruturado, persistência em PostgreSQL, defesas concretas contra as principais ameaças, containerização endurecida e uma *pipeline* GitLab CI/CD que bloqueia automaticamente código e dependências inseguros resulta em um artefato reproduzível e aderente aos princípios de DevSecOps, adequado tanto à avaliação acadêmica quanto ao uso como projeto de portfólio profissional.

9. Lições aprendidas

- Automatizar a segurança (SAST, SCA e DAST) na *pipeline* transfere a detecção de falhas para o momento mais barato de corrigi-las.
- Defesa em profundidade importa: nenhum controle isolado é suficiente, e camadas redundantes contêm falhas individuais.
- Observabilidade é parte da segurança: sem logs estruturados e detecção de anomalias, ataques como força bruta passam despercebidos.
- Containerizar com usuário não-*root* e mínimo de privilégios reduz significativamente o impacto de uma eventual exploração.

Autor: «seu nome completo» • **Repositório:** gitlab.com/«seu-usuario»/taskguard • **Licença** MIT